



Sistemas Distribuidos

Grado en Ingeniería Informática

Entrega Final.

Servicio de Mensajería

Grupo 81

Año académico 2025/2026

Juan Mayoral Fernández | 100524298 | 100524298@alumnos.uc3m.es

Jose Barrio Bielsa | 100522205 | 100522205@alumnos.uc3m.es

ÍNDICE

Descripción del código	3
Compilación y ejecución	6
Batería de pruebas	7
Conclusiones	12

Descripción del código

Servidor (server.c)

El **servidor** se encarga de recibir todas las peticiones de los **clientes** mediante la **creación de hilos por petición bajo demanda**. La comunicación con los **clientes** se lleva a cabo mediante **sockets TCP** con una **comunicación para cada petición**.

Las principales funciones del **servidor** son:

Nombre	Descripción
main	inicializa el socket de escucha , maneja la interrupción de señal y crea los hilos que llevan a cabo la función worker
worker	recibe como parámetro una estructura con el descriptor de socket y la ip del cliente , lee del socket el nombre de la operación a realizar, realiza la operación con el procedimiento que corresponda
send_message	toma de la base de datos la ip del usuario receptor , se conecta al socket del thread de escucha del cliente y envía los datos correspondientes a la operación de envío mediante la función sock_send
notify_message	(solo se realiza si el emisor está conectado) toma la ip del emisor y se conecta al socket del thread de escucha y envía el ACK mediante sock_send
send_message_attach	igual que send_message pero incluyendo el nombre del fichero adjunto
notify_message_attach	igual que notify_message pero mandando el ACK correspondiente a esta operación
logs	imprime por pantalla los mensajes correspondientes según la operación realizada, recibe como parámetros el nombre de la operación , el nombre de usuario que la pide y el resultado (OK FAIL)
message_logs	imprime por pantalla los mensajes correspondientes al envío de mensajes , distingue entre mensajes enviados a usuarios conectados o mensajes almacenados en la base de datos
sock_send sock_receive	ambas entran en bucle en el que utilizan la función write sobre el descriptor de socket hasta que se envía/recibe todo u ocurre un error
call_log	llama al servidor RPC para registrar las operaciones finalizadas correctamente

Cabe destacar la utilización de la librería **cJSON** para la implementación de la **base de datos**. El archivo **database.json** tiene la siguiente estructura:

```
JSON
{
    "<userName>": {
        "conn":          <0/1>,
        "id":            <id>,
        "num_pending":   <num_pending>,
        "pending": {
            "<sName>,<id>,<file>": <mssg>,
            ...
        }
    }
}
```

Esta librería fue tomada de <https://github.com/DaveGamble/cJSON.git> desde la que descargamos los archivos **cJSON.h** y **cJSON.c**.

Mediante las funciones **toFile** y **fromFile** en **list.h** podemos utilizar distintas estructuras para **almacenar** y para **modificar** los datos siguiendo el siguiente flujo:

Función	Transformaciones
toFile	lista enlazada → árbol cJSON → archivo.json
fromFile	archivo.json → árbol cJSON → lista enlazada

Para proteger la **base de datos** de la amenaza de la **condición de carrera** presentada por el uso de **threads** hemos utilizado un mutex global **mutex_db** con las operaciones **lock/unlock** antes y después de cada operación de **list.h**

Cliente (client.py)

El **cliente** parte del código base proporcionado en **Aula Global** en el que un **hilo principal** gestiona la entrada por **consola** y la **operación** correspondiente a la entrada y un **hilo secundario** creado al realizar la operación **CONNECT** que recibe mensajes enviados por el **servidor**.

La mayoría de las funciones del **cliente** se no dieron implementadas y nuestro trabajo consistió en implementar las funciones referentes a las **operaciones** del **cliente** siguiendo el protocolo descrito en el enunciado.

Además, para cumplir con el requisito de implementar un **servicio web**, tomamos inspiración del **github** de la asignatura para realizar la función **preprocesado(texto)** que hace uso del servicio implementado en **webService.py**. El repositorio en cuestión era el siguiente:

[uc3m_sd/materials/topic-ws/web-services.md at main](#)

Servicio Web (webService.py)

Aquí se implementa la función **normalizar()** que se ofrece como **servicio web** mediante los paquetes **Flask** y **request** al **cliente** que los utiliza para eliminar espacios extras en los mensajes que envía.

Servicio RPC (webService.py)

Usando el modelo **ONC-RPC**, generamos a partir de **logger.x** y el comando **rpcgen** todos los archivos necesarios para implementar el registro de operaciones pedido en el enunciado. En **logger.x** introducimos la estructura **log_struct** que almacena: **1.** el nombre de usuario que realiza la operación, **2.** el nombre de la operación realizada, **3.** el nombre del fichero enviado (si lo hubiera). De esta forma, la función **log** solamente tiene como único parámetro esta estructura. Este servicio es utilizado por el **servidor** tras realizar cada operación **call_log()**.

Compilación y ejecución

Hemos creado un **Makefile** para poder compilar todo el proyecto de manera más eficiente. Por eso basta con ejecutar **make clean** (por si acaso) y luego **make** para tener el proyecto listo para **ejecutar**. En este archivo hemos modularizado todos los **objetos** necesarios para cada **ejecutable** y también para la **generación** y posterior **compilación** de los **objetos rpc**.

Para ejecutar el proyecto necesitaremos **2 terminales** (mínimo): una para cada **cliente** y otra para el **servidor**:

Servidor
<pre>./logger_server & export LOG_RPC_IP=localhost ./server -p 8080</pre>

Cliente N
<pre>python3 ./webService.py & python3 ./client.py -s localhost -p 8080</pre>

Batería de pruebas

Para probar el protocolo del servidor hemos desarrollado **test.py**. Creamos una clase auxiliar **myClient** que toma un **nombre de usuario** y un **puerto** y actúa como el **cliente** en las pruebas. Para transmitir mensajes al servidor, implementamos las funciones **send_string**, **recv_byte** y **recv_str** que leen y escriben sobre el **descriptor de socket** dado como argumento.

Las pruebas realizadas en este script son las siguientes:

Prueba	Descripción
T00	registra un usuario de prueba 1 con nombre:"Juan" y puerto:9001
T01	registra un usuario de prueba 2 con nombre:"Jose" y puerto:9002
T02	conecta al usuario de prueba 1
T03	realiza la operación USERS
T04	"Juan" envía a "Jose" un mensaje cuando no está conectado
T05	"Jose" se conecta y recibe el mensaje pendiente de "Juan"
T06	"Juan" envía a "Jose" un mensaje con fichero adjunto
T07	comprueba que "Jose" ha recibido los mensajes correctos del srvr
T08	desconecta al usuario de prueba 2

Estas pruebas realizan la operación correspondiente y se compara el **valor devuelto** con el valor de **ejecución exitosa**. Cabe destacar que, como no se desconecta al segundo usuario, este script debe ejecutarse después de eliminar los datos de la base de datos. Como este script no comprueba la **impresión correcta** de los mensajes ni **ejecuciones fallidas**, ejecutaremos más pruebas **manualmente** para ver los outputs. Se muestran los outputs tanto del **servidor** como del **logger** porque en algunas pruebas utilizamos el comando **./logger_server &** antes de ejecutar el servidor en la misma consola

Operación	REGISTER	Resultado	OK
servidor			
s> REGISTER user1 OK user1 REGISTER			
cliente			
c> REGISTER user1 REGISTER OK			

Operación	REGISTER	Resultado	USERNAME IN USE
servidor			

```
s> REGISTER user1 FAIL
```

cliente

```
c> REGISTER user1
USERNAME IN USE
```

Operación	UNREGISTER	Resultado	OK
-----------	------------	-----------	----

servidor

```
s> UNREGISTER user1 OK
user1 UNREGISTER
```

cliente

```
c> UNREGISTER user1
UNREGISTER OK
```

Operación	UNREGISTER	Resultado	USER DOES NOT EXIST
-----------	------------	-----------	---------------------

Este error no se va a dar porque en el cliente comprobamos que el usuario esté registrado mediante la variable `_me`.

cliente

```
c> UNREGISTER user2
Error en unregister: Introduce el user como un string...
```

Operación	CONNECT	Resultado	OK
-----------	---------	-----------	----

servidor

```
s> CONNECT user1 OK
user1 CONNECT
```

cliente

```
c> CONNECT user1
CONNECT OK
```

Operación	CONNECT	Resultado	OK
-----------	---------	-----------	----

servidor

```
s> CONNECT user2 OK
```

cliente

```
c> CONNECT user2
CONNECT OK
c> MESSAGE 1 FROM user1
prueba
END
```

Operación	CONNECT	Resultado	USER DOES NOT EXIST
servidor			
s> CONNECT user2 FAIL			
cliente			
c> CONNECT user2 CONNECT FAIL, USER DOES NOT EXIST			

Operación	CONNECT	Resultado	USER ALREADY CONNECTED
servidor			
s> CONNECT user1 FAIL			
cliente			
c> CONNECT user1 USER ALREADY CONNECTED			

Operación	DISCONNECT	Resultado	OK
servidor			
s> DISCONNECT user1 OK			
cliente			
c> DISCONNECT user1 DISCONNECT user1 DISCONNECT OK			

Operación	DISCONNECT	Resultado	USER DOES NOT EXIST
Este error no se va a dar porque en el cliente comprobamos que el usuario esté conectado mediante la variable <code>_conn</code> .			
cliente			
c> DISCONNECT user2 Error en disconnect: Introduce el user como string..			

Operación	DISCONNECT	Resultado	USER NOT CONNECTED
servidor			
Este error no se va a dar porque en el cliente comprobamos que el usuario esté conectado mediante la variable <code>_conn</code> .			
cliente			
c> DISCONNECT user2			

Error en disconnect: Introduce el user como string..

Operación	USERS	Resultado	OK
servidor			
s> USERS OK			
cliente			
c> USERS CONNECTED USERS (2 users connected) OK user2 user1			

Operación	USERS	Resultado	USER IS NOT CONNECTED
servidor			
s> USERS FAIL			
cliente			
c> USERS CONNECTED USERS FAIL, USER IS NOT CONNECTED			

Operación	SEND	Resultado	OK
servidor			
s> MESSAGE 1 FROM user1 TO user2 STORED			
cliente			
c> SEND user2 prueba 127.0.0.1 - [08/May/2026 11:17:37] "POST /normalizar HTTP/1.1" 200 - SEND OK - MESSAGE 1			

Operación	SEND	Resultado	OK
servidor			
s> SEND MESSAGE 1 FROM user2 TO user1			
cliente			
c> SEND user1 prueba 127.0.0.1 - [08/May/2026 11:17:46] "POST /normalizar HTTP/1.1" 200 - SEND MESSAGE 1 FROM user2 prueba END			

Operación	SEND	Resultado	USER DOES NOT EXIST
-----------	------	-----------	---------------------

servidor
s> SEND user1 FAIL
cliente
c> send user3 prueba 127.0.0.1 - [08/May/2026 11:17:50] "POST /normalizar HTTP/1.1" 200 - SEND FAIL, USER DOES NOT EXIST

Operación	SENDATTACH	Resultado	OK
cliente1			
c> SENDATTACH user2 Hola texto fichero.txt 127.0.0.1 - [08/May/2026 11:18:26] "POST /normalizar HTTP/1.1" 200 - SENDATTACH OK MESSAGE 2			
cliente2			
MESSAGE 2 FROM user1 Hola texto END FILE fichero.txt c>			

Operación	GETFILE	Resultado	OK
cliente			
c> GETFILE user1 fichero.txt local.txt c> Nota: Comprobamos que existe			

Operación	GETFILE	Resultado	USER NOT CONNECTED
servidor			
s> USERS user2 OK			
cliente			
c> GETFILE user3 fichero.txt local.txt CONNECTED USERS (1 users connected) OK user2 FILE TRANSFER FAILED, user not connected			

Conclusiones

Para facilitar el trabajo hemos implementado algunas semánticas implícitas para facilitar el depurado de errores y la usabilidad del sistema.

Un ejemplo de ésto es la comprobación de conexión para **enviar mensajes**. Creemos que no tiene sentido que un **usuario** mande un mensaje si no está conectado. Esto lo implementamos también porque era bueno llevar un control de la **conexión del usuario** para conocer quién es el que **envía el mensaje** sin necesidad de pasar otras variables como parámetro. Esto es, como la operación **SEND** no incluye el **nombre de usuario** que envía el mensaje no podíamos saber quién lo mandaba, por eso creamos la variable **_conn** que almacena el **nombre del usuario conectado**.

También añadimos cambios en la operación **USERS** que ahora envía también **ip** y **puerto** de cada usuario conectado para que los **clientes** puedan utilizar esos **datos** en la operación **GETFILE**. Aquí se necesitan esos datos para conocer los **datos del cliente** que le **envía el fichero**.

Esta práctica ha servido para **poner en práctica** los conceptos explicados en clase, y pasar así de la teoría a la **aplicación real**.

El ejemplo de un **servicio de mensajería** es **particularmente útil** en este caso, ya que resulta natural aplicar los distintos conceptos vistos en clase. De este modo, hemos unido los conceptos de las clases teóricas y los laboratorios en un **pequeño proyecto funcional**.



Entrega Final.

Servicio de Mensajería
